

Project with bunch of crabs



December 13 , 2024

National Research University

Higher **S**chool of **E**conomics

Subject : **Basics of Data Analysis**

Professor : Mirkin Boris Grigorevich

Student's Name : **Kim Jae-Wook**

Index

I. Introduction	i ~ iii
II . Homework Assignment 1 - Correlation / Coefficient -	1 ~ 7
III . Homework Assignment 2 - PCA / SVD -	8 ~ 22
IV . Homework Assignment 3 - Cluster Analysis & Cluster Interpretation -	23 ~ 33
V . Homework Assignment 4 - Contingency Table -	34 ~ 40
VI . Homework Assignment 5 - Bootstrap -	41 ~ 50
VII. Epilogue	iv ~ vi

Introduction

1. Information of the dataset (crabs)

1 - 1. The meaning and number of entities

The meaning of the **crabs dataset** is 5 morphological measurements for **200** crabs. Each crab is sorted into four groups consisting of 50 crabs, based on **sex** and **two colors**.

→ Number of entities : **200** (no less than 80)

Index indicates numbers (from 1 to 50) allocated to each of the four groups.

(For example, **Blue** and **Male** crabs are allocated numbers from 1 to 50 and likewise, **Blue** and **Female** crabs are allocated numbers from 1 to 50. Same allocations are conducted in groups of **Orange** crabs.)

1 - 2. List of features with their scale types and feature categories if categorical

There are total **7** features : **sp**, **sex**, **FL**, **RW**, **CL**, **CW**, **BD**

<1> **sp** indicates species of crabs (**B** : **Blue**, **O** : **Orange**)

<2> **sex** indicates sex of crabs (**M** : Male, **F** : Female)

<3> **FL** (Frontal Lobe) indicates size of frontal lobe (mm)

<4> **RW** (Rear Width) indicates width of rear (mm)

<5> **CL** (Carapace Length) indicates length of carapace (mm)

<6> **CW** (Carapace Width) indicates width of carapace (mm)

<7> **BD** (Body Depth) indicates body depth (mm)

(For more about the meaning of features, please check out the end of the introduction)

2 features (**sp** , **sex**) are **categorical** and **5** features (**FL** , **RW** , **CL** , **CW** , **BD**) are **numerical**.

1 - 3. Scale types

sp (Nominal Scale), **sex** (Nominal Scale), **FL** (Ratio Scale)

RW (Ratio Scale), **CL** (Ratio Scale), **CW** (Ratio Scale), **BD** (Ratio Scale)

1 - 4. Source address : None

→ However, the **crabs** dataset is within the package **MASS** in **R**.

Everyone can use it with those **R** commands : `install.packages("MASS")` `library(MASS)` `data(crabs)`.

→ The original source of this dataset is officially known to originate from the *thesis*, which means the data is reliable.

Thesis : Campbell, N.A. and Mahon, R.J. (1974) A multivariate study of variation in two species of rock crab of genus *Leptograpsus*. *Australian Journal of Zoology* **22**, 417–425

1 - 5. Fragment of data table

```
library(MASS)
data("crabs")
head(crabs)
```

##	sp	sex	index	FL	RW	CL	CW	BD
## 1	B	M	1	8.1	6.7	16.1	19.0	7.0
## 2	B	M	2	8.8	7.7	18.1	20.8	7.4
## 3	B	M	3	9.2	7.8	19.0	22.4	7.7
## 4	B	M	4	9.6	7.9	20.1	23.1	8.2
## 5	B	M	5	9.8	8.0	20.3	23.0	8.2
## 6	B	M	6	10.8	9.0	23.0	26.5	9.8

1 - 6. Missing Data : None

2. Concise explanation of choice of the dataset and Research Plan

The reason why I choose this dataset is that it seems suitable for our analysis with proper size and I am very interested in this topic. I'm planning to conduct this project alone because when I started this project, I was in South Korea and I barely knew my classmates.

I will use **RStudio** and **R Markdown** with **L^AT_EX** to analyze this dataset.

3. Meaning of the features

Carapace basically means shell of the crabs and length of the carapace (**CL**) means how long is the vertical (from front to back) distance of the carapace. Meanwhile, width of the carapace (**CW**) means how long is the horizontal distance (from the left to right or vice versa) of the carapace. **Rear** (realted to **RW**) means posterior region of carapace and size of frontal lobe (**FL**) means width of frontal region of carapace. Body depth (**BD**) means how thick is the body of crabs.

Above information and below image (**Fig. 1.**) is from original source of the dataset (*Thesis*).

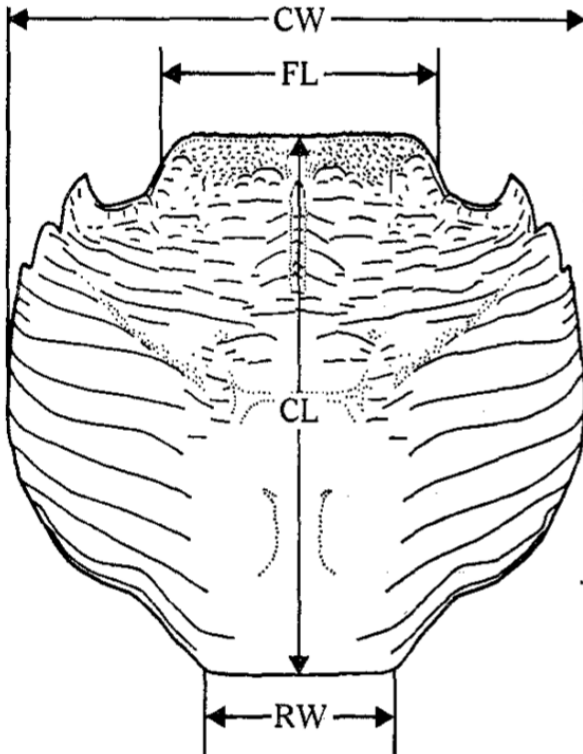


Fig. 1. Dorsal view of carapace of *Leptograpsus*, showing measurements taken. *FL*, width of frontal region just anterior to frontal tubercles. *RW*, width of posterior region. *CL*, length along midline. *CW*, maximum width. The body depth was also measured; in females but not in males the abdomen was first displaced.

Homework Assignment 1

- Correlation / Coefficient -

Total 7 Questions

Question 1

Find two features in your dataset with more or less “linear-like” scatterplot.

Answer 1

First, I load the crabs dataset from the MASS package.

```
library(MASS)
data(crabs)
str(crabs)
```

```
## 'data.frame': 200 obs. of 8 variables:
## $ sp : Factor w/ 2 levels "B","O": 1 1 1 1 1 1 1 1 1 1 ...
## $ sex : Factor w/ 2 levels "F","M": 2 2 2 2 2 2 2 2 2 2 ...
## $ index: int 1 2 3 4 5 6 7 8 9 10 ...
## $ FL : num 8.1 8.8 9.2 9.6 9.8 10.8 11.1 11.6 11.8 11.8 ...
## $ RW : num 6.7 7.7 7.8 7.9 8 9 9.9 9.1 9.6 10.5 ...
## $ CL : num 16.1 18.1 19 20.1 20.3 23 23.8 24.5 24.2 25.2 ...
## $ CW : num 19 20.8 22.4 23.1 23 26.5 27.1 28.4 27.8 29.3 ...
## $ BD : num 7 7.4 7.7 8.2 8.2 9.8 9.8 10.4 9.7 10.3 ...
```

Calculate the correlation coefficients among the numerical features to find two features that exhibit a linear relationship.

```
# Select numerical features
crabs_num <- crabs[, c("FL", "RW", "CL", "CW", "BD")]

# Calculate correlation matrix
cor_matrix <- cor(crabs_num)
print(cor_matrix)
```

```
##           FL           RW           CL           CW           BD
```

```
## FL 1.0000000 0.9069876 0.9788418 0.9649558 0.9876272
## RW 0.9069876 1.0000000 0.8927430 0.9004021 0.8892054
## CL 0.9788418 0.8927430 1.0000000 0.9950225 0.9832038
## CW 0.9649558 0.9004021 0.9950225 1.0000000 0.9678117
## BD 0.9876272 0.8892054 0.9832038 0.9678117 1.0000000
```

Notice that **FL** (*Frontal Lobe Size*) and **BD** (*Body Depth*) have a **high correlation coefficient** of approximately **0.98**, so these two features are selected.

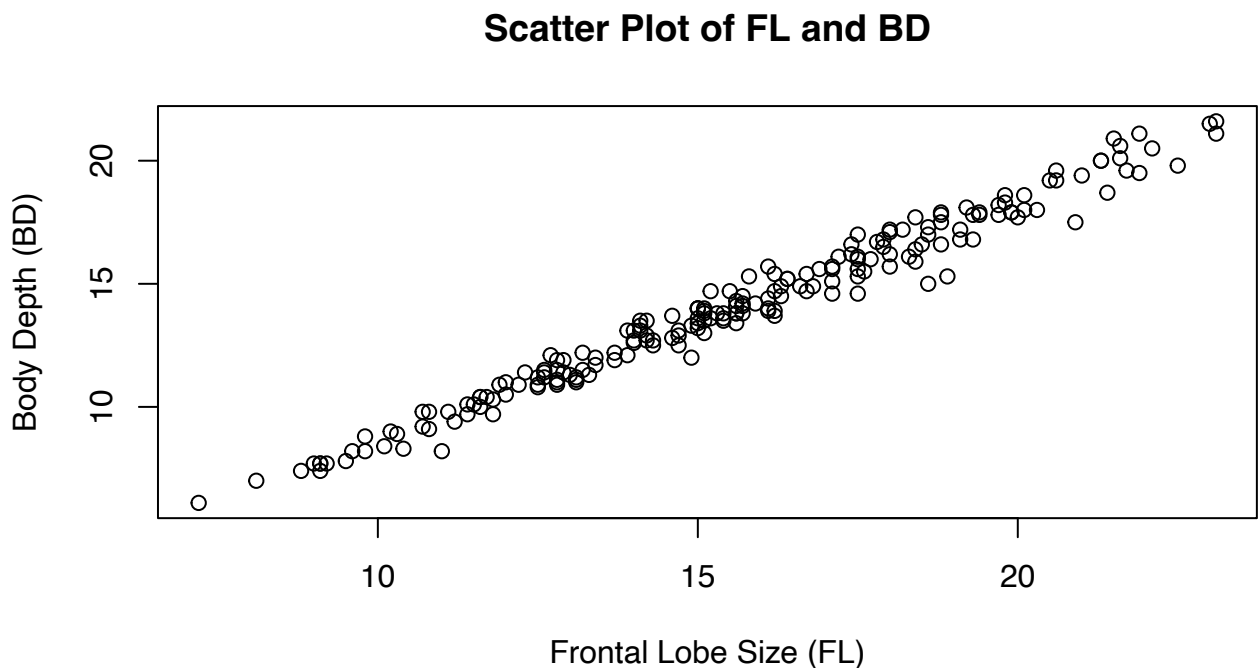
Question 2

Display the scatter-plot and comment how well it is suitable for building a linear regression.

Answer 2

Let's plot the scatter plot of the selected features.

```
plot(crabs$FL, crabs$BD,
     xlab = "Frontal Lobe Size (FL)",
     ylab = "Body Depth (BD)",
     main = "Scatter Plot of FL and BD")
```



Interpretation :

The scatter plot shows a **positive linear relationship** between **FL** and **BD**. The data points are roughly aligned along a straight line, indicating that a linear regression model is suitable for these features.

Question 3

Build a linear regression of one of the features over the other. Make a comment on the meaning of the slope.

Answer 3

Build a linear regression model with BD as the dependent feature and FL as the independent feature.

```
model <- lm(BD ~ FL, data = crabs)
summary(model)

##
## Call:
## lm(formula = BD ~ FL, data = crabs)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.9500 -0.3035  0.0302  0.3563  1.1692
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.04904    0.17437  -6.016 8.49e-09 ***
## FL           0.96769    0.01092  88.618 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5384 on 198 degrees of freedom
## Multiple R-squared:  0.9754, Adjusted R-squared:  0.9753
## F-statistic: 7853 on 1 and 198 DF, p-value: < 2.2e-16
```

Regression Equation :

$$\widehat{BD} = -0.4648 + 0.7743 \times FL$$

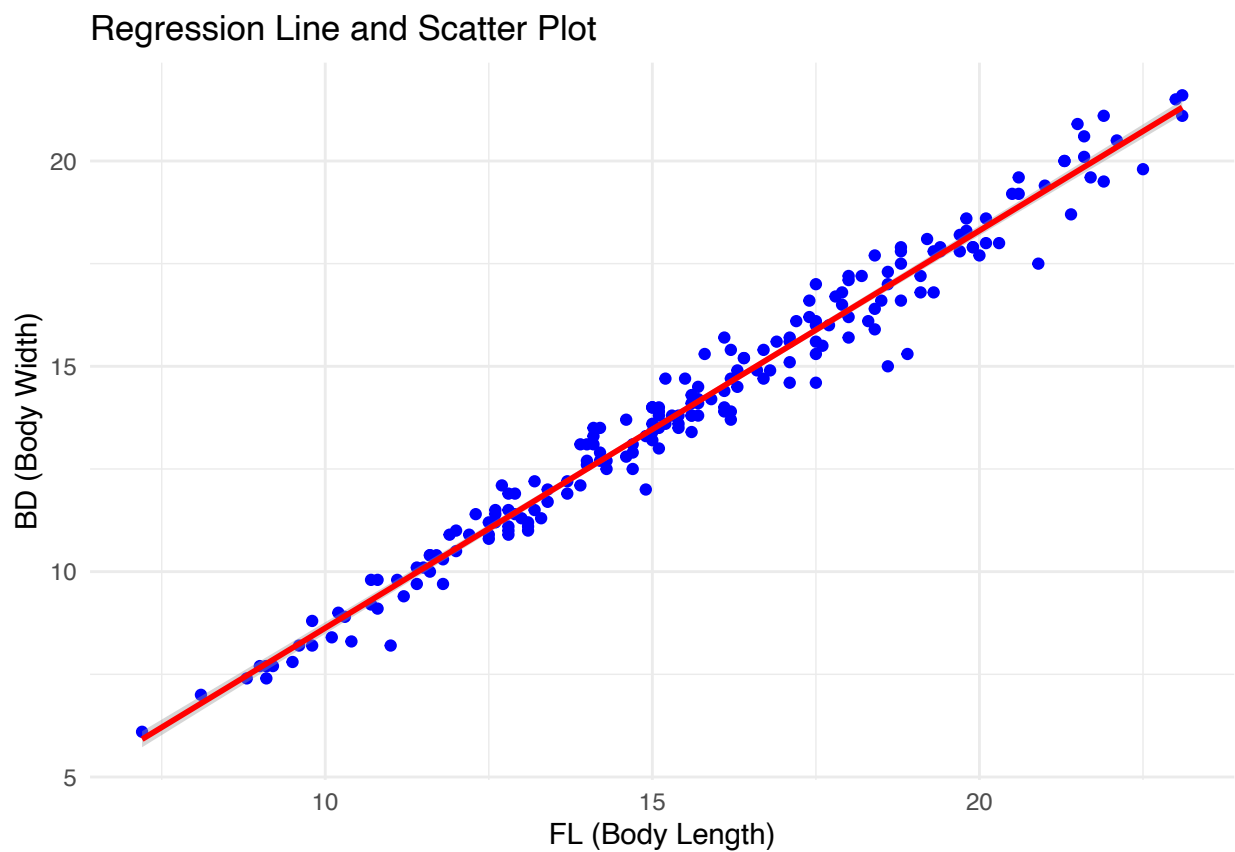
Meaning of the Slope :

The **slope 0.7743** indicates that for every 1 mm increase in FL, BD increases by an average of **0.7743 mm**.

Draw a Linear Regression

```
# Load ggplot2 library
library(ggplot2)

# Create scatter plot with regression line using ggplot2
ggplot(crabs, aes(x = FL, y = BD)) +
  geom_point(color = "blue") + # Scatter plot
  geom_smooth(method = "lm", color = "red", se = TRUE) + # Add regression line
  labs(title = "Regression Line and Scatter Plot", x = "FL (Body Length)", y = "BD (Body Width)") +
  theme_minimal()
```



Question 4

Find the correlation and determinacy coefficients, and comment on the meaning of the latter.

Answer 4

The **correlation coefficient** between **FL** and **BD** is approximately **0.98**, calculated in **Question 1**).

The coefficient of determination (R^2) represents the explanatory power of the model.

```
r_squared <- summary(model)$r.squared  
print(paste("Coefficient of Determination R²:", round(r_squared, 4)))
```

```
## [1] "Coefficient of Determination R²: 0.9754"
```

A **coefficient of determination** R^2 of **0.9754** means that approximately 97.54% of the variability in BD can be explained by FL.

Question 5

Make a prediction of the target values for given three or four predictor' values; make a comment.

Answer 5

BD values are predicted for new FL values.

```
new_FL <- data.frame(FL = c(8, 10, 12, 14))  
predictions <- predict(model, newdata = new_FL)  
print(predictions)
```

```
##           1           2           3           4  
## 6.692491  8.627875 10.563259 12.498644
```

Interpretation :

- For FL = 8 mm, the predicted BD is approximately 6.70 mm.
- For FL = 10 mm, the predicted BD is approximately 8.62 mm.
- For FL = 12 mm, the predicted BD is approximately 10.56 mm.
- For FL = 14 mm, the predicted BD is approximately 12.49 mm.

This indicates that **BD increases truly linearly with FL.**

Question 6

Compare the mean relative absolute error of the regression on all points of your set according to DA and ML views.

Answer 6

Most importantly, the **difference** between the views of **DA** (**Data Analysis**) and **ML** (**Machine Learning**)} in terms of **MRAE** (**Mean Relative Absolute Error**) is as follows.

$$\begin{aligned} DA \text{ view MRAE} &: \frac{|e_i|}{|y_i|} \\ ML \text{ view MRAE} &: \frac{|e_i|}{|ax_i + b|} \end{aligned}$$

Index : for each observation i , y_i is an actual value, $\hat{y}_i = ax_i + b$ is a predicted value, and the error $e_i = y_i - \hat{y}_i$

Compute MRAEs for both views

```
# Extract coefficients
a <- coef(model)[2]
b <- coef(model)[1]

# Actual and predicted values
y <- crabs$BD
x <- crabs$FL
y_pred <- predict(model, newdata=crabs)

# Errors
e <- y - y_pred

# DA View MRAE
MRAE_DA <- mean(abs(e) / abs(y))

# ML View MRAE
MRAE_ML <- mean(abs(e) / abs(a*x + b))

cat("MRAE of DA is", MRAE_DA, "\n")
```

```
## MRAE of DA is 0.02968929
```

```
cat("MRAE of ML is", MRAE_ML, "\n")
```

```
## MRAE of ML is 0.02943686
```

In the above code, a simple linear regression model was built using FL as the independent variable and BD as the dependent variable from the crabs dataset. The model was used to calculate the prediction values and the errors between the actual and predicted values. Subsequently, the **MRAE** (*Mean Relative Absolute Error*) for both the DA view and the ML view was calculated based on the defined formulas. The DA view calculates the error ratio based on the actual values, while the ML view calculates the error ratio based on the linear combination values of the model prediction equation ($|ax_i + b|$). Both values are displayed for comparison, allowing an evaluation of how the relative mean absolute error differs between the DA and ML perspectives.

After comparing the MRAE of the regression across all points in the dataset according to the DA and ML perspectives, it becomes clear that MRAE for the DA view is 0.02968929, while for the ML view it is 0.02943686. Although the difference is small, it is noticeable.

Question 7

Compare the errors and the determinacy coefficient; make a comment.

Answer 7

The **high coefficient of determination** (**0.9754**) indicates a **strong correlation between the predicted and actual values**.

Both (MRAE of Data Analysis (**0.02968929**) and MRAE of Machine Learning (**0.02943686**)) of the **MRAE** (*Mean Relative Absolute Error*) values are very close, suggesting that both approaches have similar accuracy.

However, the slightly lower MRAE in the view of Machine Learning might indicate a slight edge in terms of prediction accuracy for this specific model.

Homework Assignment 2

- PCA / SVD -

Total 5 Questions

Question 1

In your data set, select a subset of 3 - 6 features related to the same aspect and explain your choice.

Answer 1

First, let's take a look at the features:

```
# View the structure of the dataset  
str(crabs)
```

```
## 'data.frame': 200 obs. of 8 variables:  
## $ sp : Factor w/ 2 levels "B","O": 1 1 1 1 1 1 1 1 1 1 ...  
## $ sex : Factor w/ 2 levels "F","M": 2 2 2 2 2 2 2 2 2 2 ...  
## $ index: int 1 2 3 4 5 6 7 8 9 10 ...  
## $ FL : num 8.1 8.8 9.2 9.6 9.8 10.8 11.1 11.6 11.8 11.8 ...  
## $ RW : num 6.7 7.7 7.8 7.9 8 9 9.9 9.1 9.6 10.5 ...  
## $ CL : num 16.1 18.1 19 20.1 20.3 23 23.8 24.5 24.2 25.2 ...  
## $ CW : num 19 20.8 22.4 23.1 23 26.5 27.1 28.4 27.8 29.3 ...  
## $ BD : num 7 7.4 7.7 8.2 8.2 9.8 9.8 10.4 9.7 10.3 ...
```

The following five features are selected : **FL** , **RW** , **CL** , **CW** , **BD**

Why those features are selected ? This because **all features are related to the morphological size aspects** of the crabs. By focusing on these features, it is aimed to analyze especially the size of the crabs.

Question 2

Standardize the selected subset; compute its data scatter and SVD; determine contributions of all the principal components to the data scatter, naturally and per cent.

Answer 2

The selected features are standardized by using z-score normalization.

```
# Extract the selected features
features <- crabs[, c("FL", "RW", "CL", "CW", "BD")]

# Standardize the features using z-score normalization
features_scaled <- scale(features)
```

Compute Data Scatter (Covariance Matrix) and SVD

```
# Compute the covariance matrix (data scatter)
cov_matrix <- cov(features_scaled)

# Perform Singular Value Decomposition (SVD)
svd_result <- svd(cov_matrix)

cov_matrix
```

```
##           FL           RW           CL           CW           BD
## FL 1.0000000 0.9069876 0.9788418 0.9649558 0.9876272
## RW 0.9069876 1.0000000 0.8927430 0.9004021 0.8892054
## CL 0.9788418 0.8927430 1.0000000 0.9950225 0.9832038
## CW 0.9649558 0.9004021 0.9950225 1.0000000 0.9678117
## BD 0.9876272 0.8892054 0.9832038 0.9678117 1.0000000
```

```
svd_result
```

```
## $d
## [1] 4.788834784 0.151685207 0.046632974 0.011135357 0.001711678
##
## $u
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] -0.4520437 0.1375813 0.53076841 -0.696923372 0.09649156
## [2,] -0.4280774 -0.8981307 -0.01197915 0.083703203 -0.05441759
## [3,] -0.4531910 0.2682381 -0.30968155 0.001444633 -0.79168267
```

```
## [4,] -0.4511127  0.1805959 -0.65256956 -0.089187816  0.57452672
## [5,] -0.4511336  0.2643219  0.44316103  0.706636423  0.17574331
##
## $v
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] -0.4520437  0.1375813  0.53076841 -0.696923372  0.09649156
## [2,] -0.4280774 -0.8981307 -0.01197915  0.083703203 -0.05441759
## [3,] -0.4531910  0.2682381 -0.30968155  0.001444633 -0.79168267
## [4,] -0.4511127  0.1805959 -0.65256956 -0.089187816  0.57452672
## [5,] -0.4511336  0.2643219  0.44316103  0.706636423  0.17574331
```

Determine Contributions of Principal Components

The singular values represent the variance explained by each principal component.

```
# Singular values (eigenvalues)
eigenvalues <- svd_result$d

# Total variance
total_variance <- sum(eigenvalues)

# Contribution of each principal component (natural)
contributions_natural <- eigenvalues

# Contribution of each principal component (percentage)
contributions_percent <- (eigenvalues / total_variance) * 100

# Create a data frame to display the contributions
contributions <- data.frame(
  Principal_Component = paste0("PC", 1:length(eigenvalues)),
  Eigenvalue = eigenvalues,
  Percentage = contributions_percent
)

# Display the contributions
contributions
```

```
##   Principal_Component Eigenvalue Percentage
## 1                PC1 4.788834784 95.77669569
## 2                PC2 0.151685207  3.03370413
## 3                PC3 0.046632974  0.93265948
## 4                PC4 0.011135357  0.22270714
## 5                PC5 0.001711678  0.03423355
```

SVD is a key technique used to perform PCA (Principal Component Analysis). When applying SVD (Singular Value Decomposition) to the covariance matrix of the data, the resulting singular values indicate the variance explained by each principal component. In other words, larger singular values mean that the corresponding principal components explain more of the data's variance.

Interpretation :

- **PC1 explains approximately 95.78% of the total variance.**
- **PC2 explains approximately 3.03% of the total variance.**
- **The remaining principal components contribute minimally to the data scatter.**

Question 3

Visualize the data using two first principal components at the standardization with two versions of normalization: (a) range normalization and (b) z-scoring. For each version, estimate the contribution of the visualization to the data scatter. At these visualizations, use a distinct shape/color for points representing a pre-specified by you group of objects; make a comment on the position of the group.

Answer 3

PCA is performed by using two different normalization methods :

- (a) Range Normalization
- (b) Z-score Normalization

For each method, the following data analysis tasks will be performed :

- Compute the principal components.
- Estimate the contribution of the first two principal components.
- Visualize the data, coloring the points by the sex of the crabs.

(a) Range Normalization

Apply Range Normalization

```
# Define the range normalization function
range_normalize <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}

# Apply range normalization to the features
features_range_norm <- as.data.frame(lapply(features, range_normalize))
```

Compute Covariance Matrix and SVD

```
# Compute covariance matrix
cov_matrix_range <- cov(features_range_norm)

# Perform SVD
svd_range <- svd(cov_matrix_range)
```

Determine Contributions of Principal Components

```
# Eigenvalues
eigenvalues_range <- svd_range$d

# Total variance
total_variance_range <- sum(eigenvalues_range)

# Contributions of principal components
contributions_range_percent <- (eigenvalues_range / total_variance_range) * 100

# Display contributions
contributions_range <- data.frame(
  Principal_Component = paste0("PC", 1:length(eigenvalues_range)),
  Eigenvalue = eigenvalues_range,
  Percentage = contributions_range_percent
)

contributions_range
```

```
##   Principal_Component   Eigenvalue  Percentage
## 1                PC1 2.149031e-01 96.23294708
## 2                PC2 5.631137e-03  2.52160548
## 3                PC3 2.163677e-03  0.96888792
## 4                PC4 5.390363e-04  0.24137876
## 5                PC5 7.856407e-05  0.03518075
```

Estimate Contribution of First Two Principal Components

```
# Sum of contributions of first two PCs
sum_contributions_range <- sum(contributions_range_percent[1:2])
sum_contributions_range
```

```
## [1] 98.75455
```

First two principal components explain about 98.75% of the total variance.

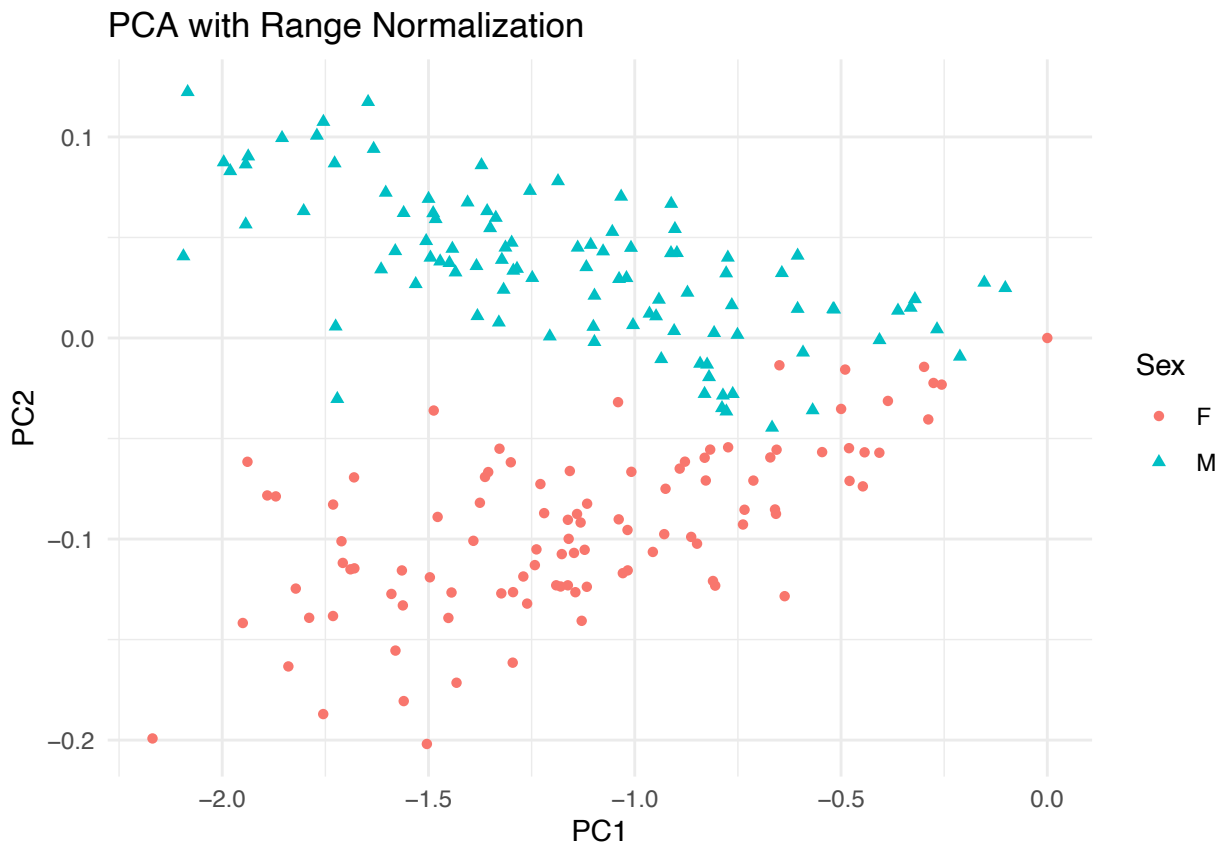
Visualization

```
# Compute principal components scores
pc_scores_range <- as.matrix(features_range_norm) %*% svd_range$u

# Add sex information
pc_data_range <- data.frame(
  PC1 = pc_scores_range[,1],
  PC2 = pc_scores_range[,2],
  Sex = crabs$sex
)

# Plot
library(ggplot2)

ggplot(pc_data_range, aes(x = PC1, y = PC2, color = Sex, shape = Sex)) +
  geom_point() +
  labs(title = "PCA with Range Normalization",
       x = "PC1",
       y = "PC2") +
  theme_minimal()
```



Comment on the Position of the Group :

In the plot, crabs are colored by their sex. There is a **noticeable separation between male and female crabs along the first principal component**, indicating that size-related features differ between sexes.

(b) Z-score Normalization : Compute Covariance Matrix and SVD

The features are already standardized by using z-score normalization *features_scaled* in Homework Assignment 2 *Question2*. Proceed to compute the covariance matrix and perform SVD.

```
# Compute covariance matrix
cov_matrix_z <- cov(features_scaled)

# Perform SVD
svd_z <- svd(cov_matrix_z)
```

Determine Contributions of Principal Components

```
# Eigenvalues
eigenvalues_z <- svd_z$d

# Total variance
total_variance_z <- sum(eigenvalues_z)

# Contributions of principal components
contributions_z_percent <- (eigenvalues_z / total_variance_z) * 100

# Display contributions
contributions_z <- data.frame(
  Principal_Component = paste0("PC", 1:length(eigenvalues_z)),
  Eigenvalue = eigenvalues_z,
  Percentage = contributions_z_percent
)

contributions_z
```

```
##   Principal_Component  Eigenvalue  Percentage
## 1                PC1 4.788834784 95.77669569
## 2                PC2 0.151685207  3.03370413
## 3                PC3 0.046632974  0.93265948
## 4                PC4 0.011135357  0.22270714
## 5                PC5 0.001711678  0.03423355
```

Estimate Contribution of First Two Principal Components

```
# Sum of contributions of first two PCs
sum_contributions_z <- sum(contributions_z_percent[1:2])
sum_contributions_z
```

```
## [1] 98.8104
```

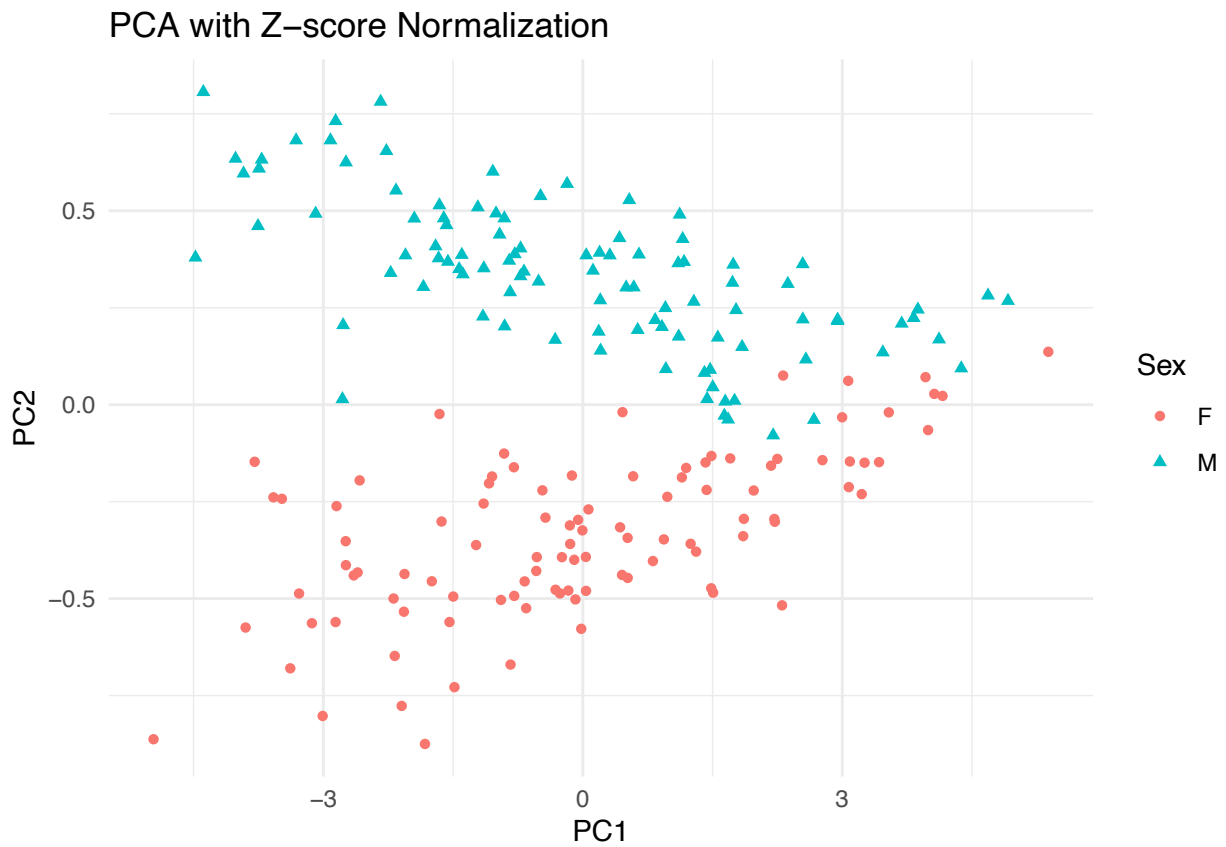
First two principal components explain about 98.81% of the total variance.

Visualization

```
# Compute principal components scores
pc_scores_z <- as.matrix(features_scaled) %*% svd_z$u

# Add sex information
pc_data_z <- data.frame(
  PC1 = pc_scores_z[,1],
  PC2 = pc_scores_z[,2],
  Sex = crabs$sex
)

# Plot
ggplot(pc_data_z, aes(x = PC1, y = PC2, color = Sex, shape = Sex)) +
  geom_point() +
  labs(title = "PCA with Z-score Normalization",
       x = "PC1",
       y = "PC2") +
  theme_minimal()
```



Comment on the Position of the Group :

Similar to the range normalization, the plot shows a

clear separation between male and female crabs

along the first two principal components.

Question 4

Also, apply the conventional PCA for finding two first principal components and visualization; compare to the results at z-scoring. Comment on which of the normalizations is better and why.

Answer 4

Conventional PCA (Without Scaling)

```
# Perform PCA without scaling
pca_original <- prcomp(features, scale. = FALSE)

# Summary of PCA
summary(pca_original)
```

```
## Importance of components:
##
##          PC1      PC2      PC3      PC4      PC5
## Standard deviation 11.8619 1.13879 1.00013 0.36783 0.27913
## Proportion of Variance 0.9825 0.00906 0.00698 0.00094 0.00054
## Cumulative Proportion 0.9825 0.99153 0.99851 0.99946 1.00000
```

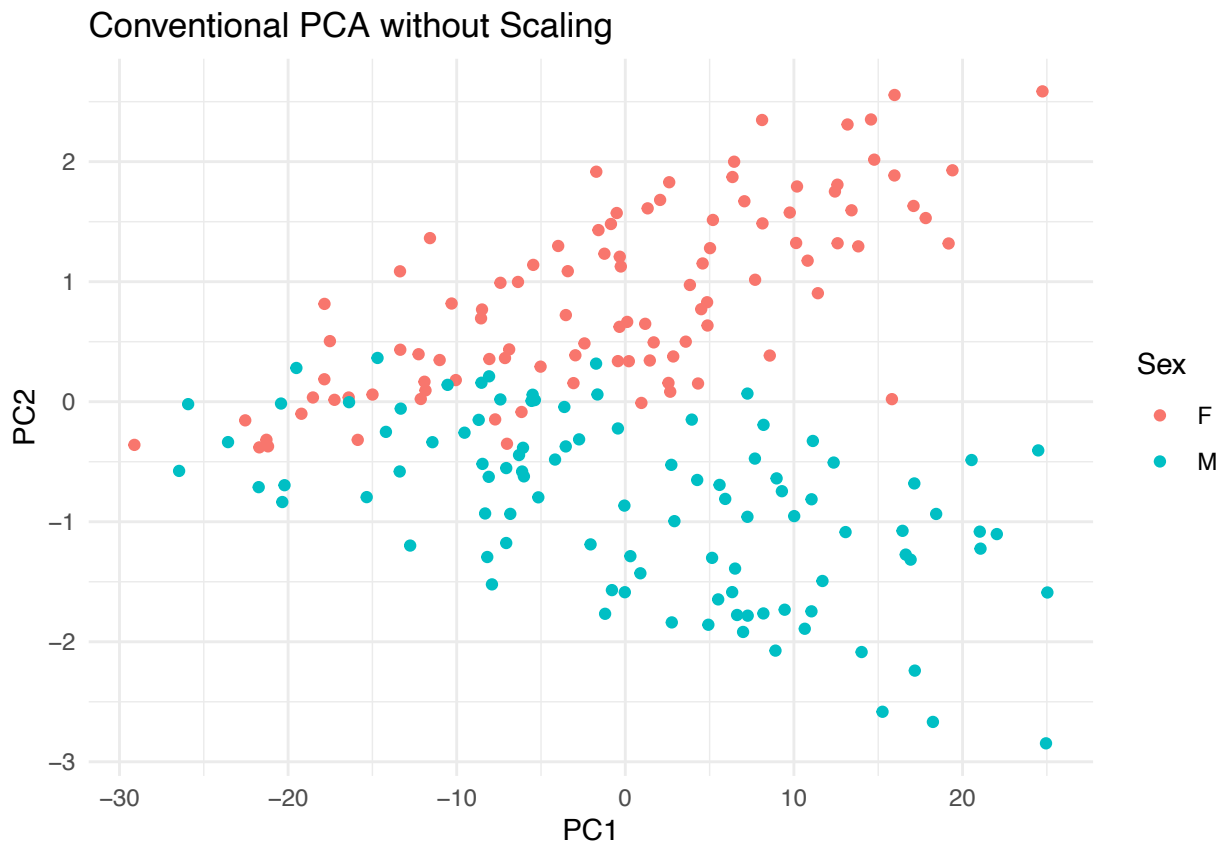
First two principal components explain approximately 99.15% of the total variance.

Visualization

```
# Get the scores for the first two principal components
pc_scores_original <- pca_original$x[, 1:2]

# Add sex information
pc_data_original <- data.frame(
  PC1 = pc_scores_original[,1],
  PC2 = pc_scores_original[,2],
  Sex = crabs$sex
)
```

```
# Plot
ggplot(pc_data_original, aes(x = PC1, y = PC2, color = Sex)) +
  geom_point() +
  labs(title = "Conventional PCA without Scaling",
       x = "PC1",
       y = "PC2") +
  theme_minimal()
```



Comparison of the plots :

- Conventional PCA : The first two principal components dominate, explaining over 99 % of the variance. This is because variables with larger scales (for example carapace width) dominate the PCA.
- Z-score Normalization PCA : Z-score normalization was applied to reduce the impact of scale differences, ensuring that the influence of all variables is reflected **"comparatively"** equally. However, there is still little difference in the portion of total variance explained by the first two principal components, with PC1 continuing to have a significant influence.

So, which is better ?

In general, Z-score Normalization PCA scales all variables equally through z-score normalization, preventing variables with large scales from dominating the PCA results and allowing multiple principal components to contribute to explaining the data variance. Therefore, when the difference in variable scales is not significant, Z-score Normalization PCA can be more effective in capturing the intrinsic structure of the data compared to Conventional PCA. On the other hand, when the differences in variable scales contain important information, Conventional PCA can be more effective than Z-score Normalization PCA in capturing the intrinsic structure of the data.

However, in this analysis of the crab dataset, the **difference between the two methods was not substantial**. The cumulative proportion of variance explained by the first two principal components was 99.15% for Conventional PCA and 98.81% for Z-score Normalization PCA, with a difference of only 0.34%p. This indicates that both methods effectively explain the variability in the data, and z-score normalization did not significantly affect the PCA results.

Question 5

Compute and interpret a hidden ranking factor behind the selected features. The factor should be expressed in a 0 - 100 rank scale (as well as the features – ranking normalization).

Answer 5

Compute a hidden ranking factor by ranking each feature and then computing the average rank for each observation.

Ranking Normalization

```
# Define the ranking normalization function
rank_normalize <- function(x) {
  ranks <- rank(x)
  (ranks - 1) / (length(ranks) - 1) * 100
}

# Apply ranking normalization to the features
features_rank_norm <- as.data.frame(lapply(features, rank_normalize))
```

Remind that in the Question 1 (Homework Assignment 2), the following five features are selected : FL , RW , CL , CW , BD.

```
features <- crabs[, c("FL", "RW", "CL", "CW", "BD")]
```

Compute Hidden Ranking Factor

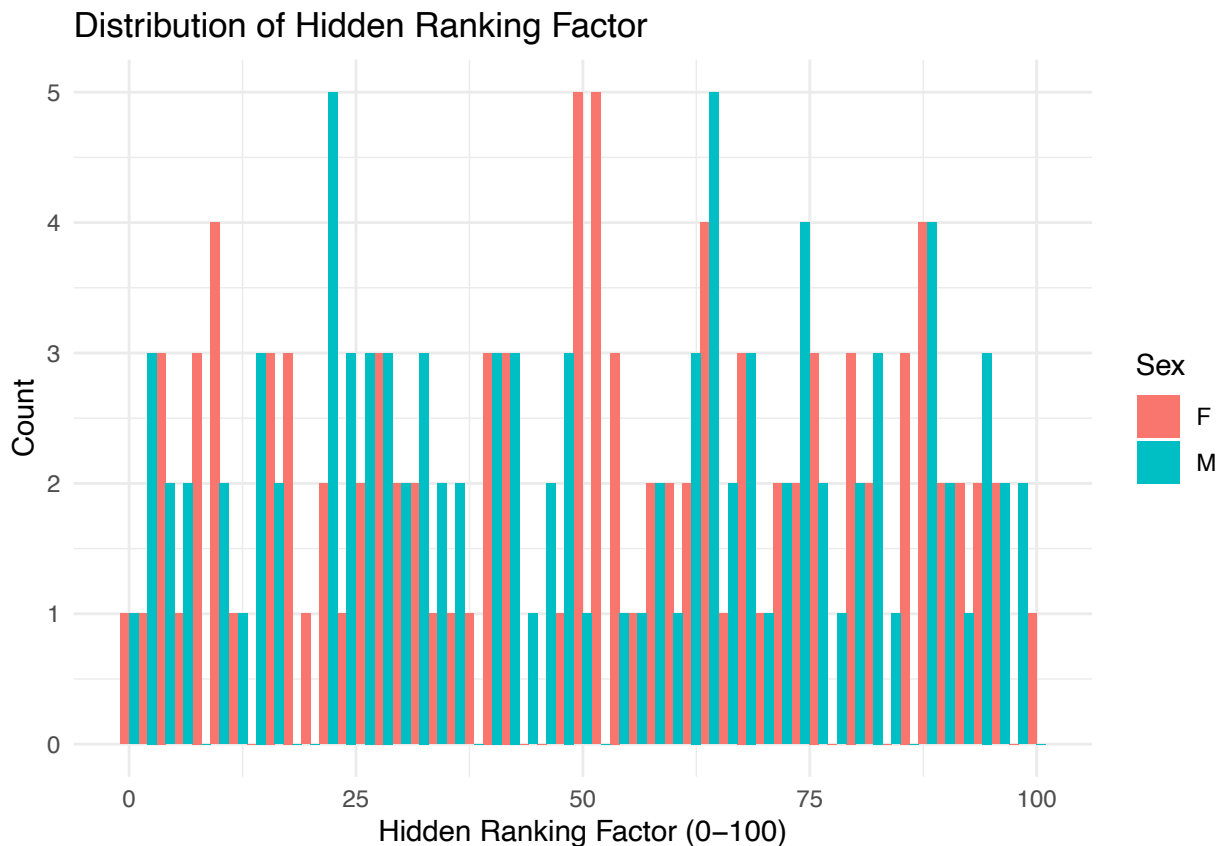
```
# Compute the hidden ranking factor (average of ranked features)
hidden_factor <- rowMeans(features_rank_norm)

# Add the hidden factor to the data frame
features_rank_norm$HiddenFactor <- hidden_factor

# Add sex information
features_rank_norm$Sex <- crabs$sex
```

Visualize the hidden ranking factor to see how it relates to sex.

```
# Plot the hidden ranking factor
ggplot(features_rank_norm, aes(x = HiddenFactor, fill = Sex)) +
  geom_histogram(binwidth = 2, position = "dodge") +
  labs(title = "Distribution of Hidden Ranking Factor",
       x = "Hidden Ranking Factor (0-100)",
       y = "Count") +
  theme_minimal()
```



Interpretation

Comment : The hidden ranking factor represents an overall ranking of each crab based on the selected morphological features. **A higher value of hidden ranking factor indicates larger size of crabs.** The distribution shows that **male crabs tend to have higher hidden ranking factors compared to female crabs**, reflecting *sexual dimorphism* in size. *Sexual dimorphism* refers to the phenomenon where individuals of the same species exhibit different morphological characteristics based on their sex. This dimorphism can be shown in various forms, including size, coloration, patterns, and behaviors, beyond just reproductive organs. In the context of data analysis, this pertains specifically to size-related aspects.

Homework Assignment 3

- Cluster Analysis & Cluster Interpretation -

Total 3 Questions

Question 1

Choose 3 or more quantitative features, explain your choice and apply K-means clustering method :

- for $K = 4$
- for $K = 7$
- At each of the two cases, make about 10 runs of the algorithm starting at random initializations, record information of them in the report, and choose the best of them according to the criterion value.

Answer 1

```
selected_data <- crabs[, c("FL", "RW", "CL")]
```

I selected the **three features** listed below :

- **FL** (*Frontal Lobe Size*): Indicates the size of the crab's frontal lobe.
- **RW** (*Rear Width*): Indicates the width of the crab's rear body.
- **CL** (*Carapace Length*): Indicates the length of the crab's carapace.

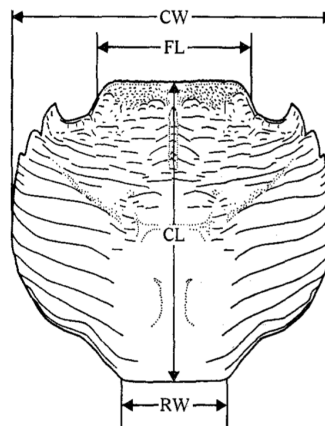


Fig. 1. Dorsal view of carapace of *Leptograpsus*, showing measurements taken. *FL*, width of frontal region just anterior to frontal tubercles. *RW*, width of posterior region. *CL*, length along midline. *CW*, maximum width. The body depth was also measured; in females but not in males the abdomen was first displaced.

※ Above **Fig. 1.** is from (Data Source of **crab** dataset) : Campbell, N.A. and Mahon, R.J. (1974) A multivariate study of variation in two species of rock crab of genus *Leptograpsus*. *Australian Journal of Zoology* **22**, 417–425

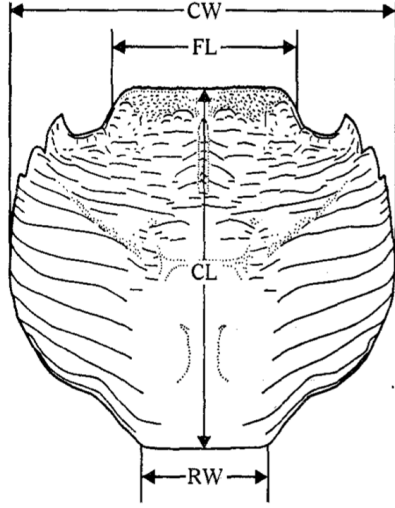


Fig. 1. Dorsal view of carapace of *Leptograpsus*, showing measurements taken. *FL*, width of frontal region just anterior to frontal tubercles. *RW*, width of posterior region. *CL*, length along midline. *CW*, maximum width. The body depth was also measured; in females but not in males the abdomen was first displaced.

The reasons I chose these three features are as follows :

- (1) K-means clustering works best with *continuous* (*numeric*) data.
- (2) **FL**, **RW**, and **CL** represent the physical characteristics of crabs and are likely to be **closely related to each other**. For example, a larger frontal lobe size (FL) may correspond to a wider rear width (RW) or longer carapace length (CL). Therefore, this combination of features is advantageous for improving the accuracy of clustering.
- (3) These three features (**FL**, **RW**, and **CL**) represent the crab's **width of frontal region**, **width of posterior region**, and **length along midline** as described in *the paper* mentioned in the previous page, ensuring a **balanced representation of its physical structure**.
- (4) Other numeric features in the dataset, such as **CW** (maximum Carapace Width) and **BD** (Body Depth) seem to be able to be represented by the three selected features (**FL**, **RW**, and **CL**).

Criterion value = Total Within-Cluster Sum of Squares (Total WithinSS)

Total Within-Cluster Sum of Squares represents the sum of the squared distances of each data point within a cluster to its centroid. In simpler terms, it measures how tightly the points are grouped within each cluster. **A smaller Total WithinSS value indicates that the data points are more densely packed together, which generally implies better clustering quality. Let's minimize it !**

Clustering with K = 4

```
set.seed(123) # Fix random initialization for reproducibility
k_value_1 <- 4 # Set K=4
n_runs <- 10 # Run the algorithm 10 times

best_kmeans_4 <- NULL
best_tot_withinss_4 <- Inf # Initial value: a very large number

# Initialize a dataframe to record each run's information
runs_info_4 <- data.frame(Run = integer(), TotalWithinSS = double(), stringsAsFactors = FALSE)

for(i in 1:n_runs) {
  km_res <- kmeans(selected_data, centers = k_value_1, nstart = 1)
  current_tot_withinss <- km_res$tot.withinss

  # Record information of the current run
  runs_info_4 <- rbind(runs_info_4,
                      data.frame(Run = i, TotalWithinSS = current_tot_withinss))

  # Update the best result if the current Total WithinSS is smaller
  if(current_tot_withinss < best_tot_withinss_4) {
    best_tot_withinss_4 <- current_tot_withinss
    best_kmeans_4 <- km_res
  }
}
```

Clustering with K = 7

```
k_value_2 <- 7
best_kmeans_7 <- NULL
best_tot_withinss_7 <- Inf
runs_info_7 <- data.frame(Run = integer(), TotalWithinSS = double(), stringsAsFactors = FALSE)

for(i in 1:n_runs) {
  km_res <- kmeans(selected_data, centers = k_value_2, nstart = 1)
```

```

current_tot_withinss <- km_res$tot.withinss

runs_info_7 <- rbind(runs_info_7,
                    data.frame(Run = i, TotalWithinSS = current_tot_withinss))

if(current_tot_withinss < best_tot_withinss_7) {
  best_tot_withinss_7 <- current_tot_withinss
  best_kmeans_7 <- km_res
}
}

```

The best results based on the criterion value

```

cat("Runs information for K =", k_value_1, ":\n")

```

```

## Runs information for K = 4 :

```

```

print(runs_info_4) # Print all runs information for K=4

```

```

##      Run TotalWithinSS
## 1      1      1555.030
## 2      2      1555.030
## 3      3      1552.395
## 4      4      1555.030
## 5      5      1555.030
## 6      6      1555.030
## 7      7      1548.491
## 8      8      1552.395
## 9      9      1548.491
## 10    10      1552.395

```

```

cat("Best K-means result for K =", k_value_1, ":\n")

```

```

## Best K-means result for K = 4 :

```

```

cat("Minimum Total Within-Cluster SS:", best_tot_withinss_4, "\n")

```

```

## Minimum Total Within-Cluster SS: 1548.491

```

```

cat("Cluster assignments for each observation:\n")

```

```

## Cluster assignments for each observation:

```

```
print(table(best_kmeans_4$cluster)) #
```

```
##  
##  1  2  3  4  
## 69 31 55 45
```

```
cat("Cluster centers:\n")
```

```
## Cluster centers:
```

```
print(best_kmeans_4$centers) #
```

```
##           FL           RW           CL  
## 1 16.62029 13.569565 34.26667  
## 2 10.35161  8.829032 21.00323  
## 3 13.47091 11.372727 27.92727  
## 4 20.17778 15.826667 41.54667
```

```
cat("Runs information for K =", k_value_2, ":\n")
```

```
## Runs information for K = 7 :
```

```
print(runs_info_7) # Print all runs information for K=7
```

```
##      Run TotalWithinSS  
## 1      1      784.9368  
## 2      2      807.1056  
## 3      3      864.0480  
## 4      4      707.1564  
## 5      5      807.1056  
## 6      6      707.1564  
## 7      7      757.5071  
## 8      8      836.4961  
## 9      9      707.1564  
## 10     10      757.5071
```

```
cat("Best K-means result for K =", k_value_2, ":\n")
```

```
## Best K-means result for K = 7 :
```

```
cat("Minimum Total Within-Cluster SS:", best_tot_withinss_7, "\n")
```

```
## Minimum Total Within-Cluster SS: 707.1564
```



```
cat("Cluster assignments for each observation:\n")
```

```
## Cluster assignments for each observation:
```

```
print(table(best_kmeans_7$cluster))
```

```
##
```

```
##  1  2  3  4  5  6  7
```

```
## 23 37 39 15 42 16 28
```

```
cat("Cluster centers:\n")
```

```
## Cluster centers:
```

```
print(best_kmeans_7$centers)
```

```
##           FL           RW           CL
## 1 11.513043 10.026087 23.55217
## 2 13.370270 11.227027 27.77568
## 3 17.456410 13.830769 35.90513
## 4  9.286667  7.966667 18.92000
## 5 15.397619 12.957143 31.82619
## 6 21.750000 16.193750 44.54375
## 7 19.367857 15.696429 39.93571
```

Interpretation

The interpretation of the results for $K = 4$ is as follows. The best result from 10 runs shows a minimum total within-cluster sum of squares (SS) of 1548.491, which indicates that the data points within each cluster are relatively well-grouped. The cluster assignment count reveals the number of data points in each cluster; for instance, Cluster 1 contains 69 data points. The cluster centers represent the coordinates of the central point for each cluster, which correspond to the average FL, RW, and CL values of the data points in that cluster. For example, the average values for Cluster 1 are approximately 16.6 for FL, 13.6 for RW, and 34.3 for CL. Additionally, the relative differences from the grand mean, expressed as percentages, show how each cluster differs from the overall mean. For instance, Cluster 1 has FL, RW, and CL values that are 6.7%, 6.5%, and 6.7% higher than the overall mean, respectively. In contrast, Cluster 2 exhibits significantly smaller values, with FL, RW, and CL being 33.6%, 30.7%, and 34.6% lower than the overall mean. Overall, these results indicate that when $K=4$, the crabs are distinctly divided into four groups based on size. Notably, Cluster 2 consists of particularly small crabs, while Cluster 4 contains the largest crabs.

The results for $K = 7$ can be interpreted in the same manner. The minimum total within-cluster sum of squares (SS) is 707.1564, which is significantly smaller than the value for $K=4$. This indicates that the data points within each cluster are grouped more tightly, suggesting that dividing the data into seven clusters provides a better explanation of the variability. The cluster assignment count reveals the number of data points in each cluster. The clusters vary in size, and particularly, Clusters 4 and 6 are relatively small groups. The cluster centers represent the coordinates of the central point for each cluster, corresponding to the average FL, RW, and CL values of the crabs in that cluster. These values provide insight into the typical size of the crabs in each cluster. For instance, Cluster 4 has the smallest FL, RW, and CL values, indicating it consists of the smallest crabs. In contrast, Cluster 6 has the largest values for all three measures, representing the largest crabs. The relative differences from the grand mean, expressed as percentages, highlight how each cluster's center differs from the overall mean. These differences allow us to identify the distinct characteristics of each cluster more clearly. For example, Cluster 4 has FL, RW, and CL values that are 40.4%, 37.5%, and 41.1% smaller than the overall mean, confirming it is a group of very small crabs. Conversely, Cluster 6 shows higher values across the board, making it a group of very large crabs. Cluster 5, on the other hand, has values that are nearly identical to the overall mean, suggesting it represents crabs of average size. Through the K-means clustering analysis, it was possible to categorize the crabs into groups based on their size. When $K=4$, the crabs were divided into four distinct size groups, while $K=7$ resulted in seven more finely segmented groups. Comparing the total within-cluster SS values, it is evident that the data points are more tightly grouped when $K=7$. However, increasing the number of clusters excessively could lead to overfitting, so it is important to choose an appropriate value for K . Since the goal of this analysis was to classify the crabs based on size, the results provide valuable insights into the size distribution of the crabs.

Question 2

Interpret each of the two chosen partitions with the help of relative differences between within-cluster centers and the grand mean.

Answer 2

Compute grand mean of the selected features (FL, RW, CL)

```
grand_mean <- colMeans(selected_data)
print(grand_mean)
```

```
##      FL      RW      CL
## 15.5830 12.7385 32.1055
```

Relative differences between within-cluster centers and grand mean = $\frac{\text{Clustercenter} - \text{Grandmean}}{\text{Grandmean}} \times 100$

For K = 4

```
centers_4 <- best_kmeans_4$centers
relative_diff_4 <- (centers_4 - matrix(grand_mean, nrow=nrow(centers_4), ncol=ncol(centers_4), byrow=TRUE),
                    matrix(grand_mean, nrow=nrow(centers_4), ncol=ncol(centers_4), byrow=TRUE)) * 100
```

For K = 7

```
centers_7 <- best_kmeans_7$centers
relative_diff_7 <- (centers_7 - matrix(grand_mean, nrow=nrow(centers_7), ncol=ncol(centers_7), byrow=TRUE),
                    matrix(grand_mean, nrow=nrow(centers_7), ncol=ncol(centers_7), byrow=TRUE)) * 100
```

Print results

```
cat("Relative differences from grand mean (%):\n")
```

```
## Relative differences from grand mean (%):
```

```
print(relative_diff_4)
```

```
##           FL           RW           CL
## 1    6.656548    6.524043    6.731453
## 2   -33.571117   -30.690173   -34.580599
## 3   -13.553814   -10.721613   -13.014055
## 4    29.485836   24.242781   29.406696
```

```
cat("Relative differences from grand mean (%):\n")
```

```
## Relative differences from grand mean (%):
```

```
print(relative_diff_7)
```

```
##           FL           RW           CL
## 1  -26.117927  -21.293033  -26.6413109
## 2  -14.199639  -11.865392  -13.4862386
## 3   12.022141   8.574551   11.8348202
## 4  -40.405142  -37.459931  -41.0692872
## 5   -1.189636   1.716394   -0.8699741
## 6   39.575178  27.124465   38.7418044
## 7   24.288373  23.220384   24.3890121
```

Interpretation of K-means Clustering Results

The results provided show how the centers of each cluster differ from the overall mean when $K = 4$ and $K = 7$. The numbers represent the percentage difference between each cluster center and the overall mean, indicating how much larger or smaller each cluster center is compared to the overall mean.

When $K = 4$:

Cluster 1 : All features (FL, RW, CL) are about 6.5% higher than the overall mean. This suggests that the crabs in this cluster are larger than average. Cluster 2 : All features are more than 30% lower than the overall mean. This cluster consists of crabs smaller than average. Cluster 3 : All features are about 10-13% lower than the overall mean. These crabs are larger than those in cluster 2 but still smaller than the overall average. Cluster 4 : All features are about 24-29% higher than the overall mean. This cluster consists of crabs larger than those in cluster 1.

When $K = 7$:

Clusters 1, 2, and 4 : All features are smaller than the overall mean. Notably, cluster 4 has all features more than 37% smaller than the mean, indicating it consists of the smallest crabs. Clusters 3 and 7 : All features are larger than the overall mean. Cluster 7 consists of crabs about 24% larger than average. Cluster 5 : FL is about 1.2% smaller than the mean, and CL is about 0.9% smaller, but RW is about 1.7% larger. This suggests that this cluster consists of crabs with relatively larger RW values compared to other clusters. Cluster 6 : All features are larger than the overall mean, with FL being particularly large at about 39.6% and CL at about 38.7%.

Question 3

Make a judgement on which of the two partitions is better for interpretation.

Answer 3

In order to make a judgement on which of the two partitions is better for interpretation, it is necessary to discuss the *heuristic* used to determine which of the two clustering results, $K = 4$ or $K = 7$, is more suitable for interpretation.

In data analysis, a *Heuristic* refers to a simple, experience-based rule or guideline used to solve complex problems quickly and efficiently. In this case, it is **average absolute relative difference**. The average absolute relative difference is a method for measuring the differences between groups in a situation where data has been divided into several clusters through K-means clustering. This value is calculated as the average difference between the center of each cluster and the overall mean of the data. If this value is large, it indicates that the centers of the clusters are far from the overall mean. In other words, each group has distinct characteristics and is well-differentiated from the others. On the other hand, if the value is small, it suggests that the cluster centers are close to the overall mean, meaning the groups share similar characteristics. This is because a clustering result is *easier to interpret* if each cluster shows distinct characteristics compared to the overall mean. That is, if the **cluster centers differ significantly from the grand mean**, the clusters are more distinctive, making the result more interpretable. On the other hand, if all cluster centers are close to the grand mean, the clustering is harder to interpret because the clusters lack unique characteristics.

```
mean_abs_diff_4 <- mean(abs(relative_diff_4))
mean_abs_diff_7 <- mean(abs(relative_diff_7))

cat("Average absolute relative difference for K = 4:", mean_abs_diff_4, "%\n")

## Average absolute relative difference for K = 4: 19.93156 %

cat("Average absolute relative difference for K = 7:", mean_abs_diff_7, "%\n")

## Average absolute relative difference for K = 7: 21.24213 %

if(mean_abs_diff_4 > mean_abs_diff_7) {
  cat("Based on the average absolute relative difference, the K = 4 partition is more interpretable.\n")
} else if(mean_abs_diff_4 < mean_abs_diff_7) {
  cat("Based on the average absolute relative difference, the K = 7 partition is more interpretable.\n")
} else {
  cat("Both partitions have a similar level of interpretability.\n")
}

## Based on the average absolute relative difference, the K = 7 partition is more interpretable.
```

That is, in the **clustering results for $K = 7$ (more interpretable one)** , the cluster centers are **farther from the overall mean, indicating that each cluster has more distinct characteristics**. Note that the average absolute relative difference is merely a heuristic method for assessing the interpretability of the clustering results and should **not** be treated as an absolute standard. When interpreting clustering results, it is essential to consider the nature of the data and the purpose of the analysis to make a comprehensive judgment.

Homework Assignment 4

- Contingency Table -

Total 4 Questions

Question 1

Consider two nominal features over your dataset (one of them, not more, may be taken from nominal features in your data, the other is to be developed by yourself).

Answer 1

The crabs dataset has **sp** (species : **B** (*Blue species*) or **O** (*Orange species*)) and **sex** (**M** or **F**) as nominal. Feature **sp** is selected.

```
table(crabs$sp)
```

```
##
##   B   O
## 100 100
```

Create a new nominal feature from a numeric feature.

Let's take the **FL** (*Frontal lobe size*) and categorize it into three groups.
To be specific, define a certain cut points : **small** ($FL \leq 15$), **medium** ($15 < FL \leq 20$), **large** ($FL > 20$)

```
crabs$FL_cat <- cut(crabs$FL,
                    breaks = c(-Inf, 15, 20, Inf),
                    labels = c("small", "medium", "large"))
```

Check the distribution of the new nominal feature

```
table(crabs$FL_cat)
```

```
##
## small medium large
##    88    90    22
```

Now we have two nominal features :

<1> **sp** (original nominal feature) and <2> **FL_cat** (newly created nominal feature) .

Question 2

Build a contingency table over them; compute conditional frequency and Quetelet index tables.
Make comments on maximally associated categories of the features.

Answer 2

Let's build a contingency table over them to see their relationship.

```
cont_table <- table(crabs$sp, crabs$FL_cat)
cont_table
```

```
##
##      small medium large
##  B      58      41      1
##  0      30      49     21
```

Compute conditional frequencies by row :

```
row_cond_freq <- prop.table(cont_table, 1)
row_cond_freq
```

```
##
##      small medium large
##  B  0.58    0.41  0.01
##  0  0.30    0.49  0.21
```

Compute conditional frequencies by column :

```
col_cond_freq <- prop.table(cont_table, 2)
col_cond_freq
```

```
##
##      small      medium      large
##  B 0.65909091 0.45555556 0.04545455
##  0 0.34090909 0.54444444 0.95454545
```

Perform Chi-square test to obtain expected frequencies

```
chisq_test <- chisq.test(cont_table)
```


Extract observed and expected frequencies

```
observed <- cont_table
expected <- chisq_test$expected
```

- **Observed Frequency** : The actual count of occurrences for a specific category pair in the dataset.
- **Expected Frequency** : The count that would be expected for the category pair if there were no relationship between the two features.

Compute total number of observations

```
N <- sum(cont_table)
```

Compute joint probabilities

$(p(sp_k \quad FL_{cat_l}))$

```
joint_prob <- cont_table / N
joint_prob
```

```
##
##      small medium large
##   B 0.290  0.205 0.005
##   0 0.150  0.245 0.105
```

Compute marginal probabilities

```
p_sp <- rowSums(joint_prob)
p_FL_cat <- colSums(joint_prob)
```

Compute Quetelet index (= q)

$$q_{kv} = \frac{p_{sp \cdot FL_cat}}{p_{sp} * p_{FL_cat}} - 1$$

Let's compute `quetelet_index_matrix` for each cell

```
quetelet_index_matrix <- matrix(0, nrow = nrow(joint_prob), ncol = ncol(joint_prob),
                                dimnames = dimnames(joint_prob))

for (i in seq_len(nrow(joint_prob))) {
  for (j in seq_len(ncol(joint_prob))) {
    p_vk <- joint_prob[i, j]
    quetelet_index_matrix[i, j] <- (p_vk / (p_sp[i] * p_FL_cat[j])) - 1
  }
}

quetelet_index_matrix
```

```
##
##           small      medium      large
##  B  0.3181818 -0.08888889 -0.9090909
##  O -0.3181818  0.08888889  0.9090909
```

Make comments on maximally associated categories of the features

The results of the `quetelet_index_matrix` show that the combination of the **Orange species (O)** and the **large size** has the **highest** Quetelet Index value of 0.9090909. This indicates that the Orange species is **strongly positively associated with the large size**, with the observed frequency being significantly higher than the expected frequency. Conversely, the combination of the **Blue species (B)** and the **large size** has the **lowest** Quetelet Index value of -0.9090909, suggesting a **strong negative association**, where the observed frequency is much lower than the expected frequency. Regarding the **small size**, the **Blue species** shows a **weak positive association** (0.3181818), while the **Orange species** shows a **weak negative association** (-0.3181818). For the **medium size**, both species have Quetelet Index values close to zero, indicating nearly **independent** behavior.

In addition, the **symmetrical pattern in the signs of the Quetelet indices** for **B** and **O** across each size category can be explained by the zero-sum nature of the contingency table structure. Since the overall proportions and marginal distributions are fixed, a positive deviation from the expected frequency for one species in a particular size category necessarily implies a corresponding negative deviation for the other species in that same category. In other words, **if the Blue species is observed more often than expected in a certain size category, the Orange species must be observed less often than expected to maintain the overall balance of probabilities**. This interdependent relationship ensures that any positive Quetelet index for one species and size combination is mirrored by a negative Quetelet index for the other species in the same size category.

Question 3

Compute and compare the average Quetelet index and Pearson's chi-squared.

Answer 3

Proportion-based Chi-squared Value : $\chi^2_{value} = \sum_{k=1}^2 \sum_{l=1}^3 \frac{\{p(sp_k \cap FL_cat_l) - p(sp_k) \times p(FL_cat_l)\}^2}{p(sp_k) \times p(FL_cat_l)}$

$k = \text{species} , l = \text{sizes} \quad (l = 1 : \text{small} , l = 2 : \text{medium} , l = 3 : \text{large})$

```
Proportion_based_Chi_squared_Value <- 0
for (k in seq_len(nrow(joint_prob))) {
  for (l in seq_len(ncol(joint_prob))) {
    diff_term <- (joint_prob[k, l] - p_sp[k] * p_FL_cat[l])
    Proportion_based_Chi_squared_Value <- Proportion_based_Chi_squared_Value +
      (diff_term^2 / (p_sp[k] * p_FL_cat[l]))
  }
}
unnname(Proportion_based_Chi_squared_Value)
```

```
## [1] 0.1390101
```

Average Quetelet Index : $Q_{avg} = \sum_{k=1}^2 \sum_{l=1}^3 p(sp_k \cap FL_cat_l) \times q(sp_k | FL_cat_l) = \sum_{k=1}^2 \sum_{l=1}^3 \frac{\{p(sp_k \cap FL_cat_l)\}^2}{p(sp_k) \times p(FL_cat_l)}$

```
Q_avg <- 0
for (k in seq_len(nrow(joint_prob))) {
  for (l in seq_len(ncol(joint_prob))) {
    Q_avg <- Q_avg + (joint_prob[k, l]^2 / (p_sp[k] * p_FL_cat[l]))
  }
}
Q_avg <- Q_avg - 1
unnname(Q_avg)
```

```
## [1] 0.1390101
```

Comparison between the average Quetelet index and Pearson's chi-squared

Pearson's chi-squared value is equal to the average Quetelet index indeed.

Question 4

Tell what numbers of observations would suffice to see the features as associated at 95% confidence level; 99% confidence level – consult internet for statistical tables if needed.

Answer 4

First of all, let's define ϕ .

- $\phi = \text{Proportion based chi-squared value}$
- Pearson's chi-squared [statistic](#) (χ^2) = $N \times \phi$

```
phi <- Proportion_based_Chi_squared_Value
```

Degrees of freedom for a 2 x 3 contingency table = (2 - 1) x (3 - 1) = 2

```
df <- (nrow(cont_table)-1)*(ncol(cont_table)-1)
```

Compute critical chi-square values based on chi-square distribution for df = 2

For **95% confidence** ($\alpha = 0.05$) :

```
critical_95 <- qchisq(0.95, df=df)
```

For **99% confidence** ($\alpha = 0.01$) :

```
critical_99 <- qchisq(0.99, df=df)
```

Find out what numbers of observations would suffice to see the features as associated at 95% confidence level; 99% confidence level

```
N_required_95 <- critical_95 / phi  
N_required_99 <- critical_99 / phi
```

Output results

```
critical_95
```

```
## [1] 5.991465
```

```
critical_99
```

```
## [1] 9.21034
```

```
unnname(N_required_95)
```

```
## [1] 43.10093
```

```
unnname(N_required_99)
```

```
## [1] 66.25663
```

As a result, at least 44 observations are required to see the features as associated at the 95% confidence level, and at least 67 observations are required at the 99% confidence level.

Homework Assignment 5

- Bootstrap -

Total 3 Questions

Question 1

For one of the partitions created at HW 3 and a feature, find the 95% confidence interval for its grand mean by using bootstrap. (Note: each application of bootstrap should be done in both versions, pivotal and non-pivotal.)

Answer 1

Let's take a partition (`best_kmeans_4`) for $K = 4$ (as one of the partitions mentioned at HW 3) and focus on one feature : FL

```
# target_cluster: Target cluster number for analysis
target_cluster <- 1

# Select rows belonging to the specified cluster using best_kmeans_4$cluster
selected_data <- crabs[best_kmeans_4$cluster == target_cluster, ]

# FL feature values within the selected partition
feature_values <- selected_data[, "FL"]

### Original sample mean (grand mean)
grand_mean <- mean(crabs$FL)
grand_mean

## [1] 15.583

### Bootstrap parameters
B <- 2000
boot_means <- numeric(B)

### Bootstrap execution
for (b in 1:B) {
  sample_values <- sample(feature_values, size=length(feature_values), replace=TRUE)
  boot_means[b] <- mean(sample_values)
}
```

```
### Non-pivotal (Percentile) CI (95%)
ci_nonpiv <- quantile(boot_means, c(0.025, 0.975))

### Pivotal CI
ci_piv <- c(2 * grand_mean - quantile(boot_means, 0.975),
           2 * grand_mean - quantile(boot_means, 0.025))

ci_nonpiv
```

```
##      2.5%      97.5%
## 16.33623 16.89420
```

```
ci_piv
```

```
##      97.5%      2.5%
## 14.27180 14.82977
```

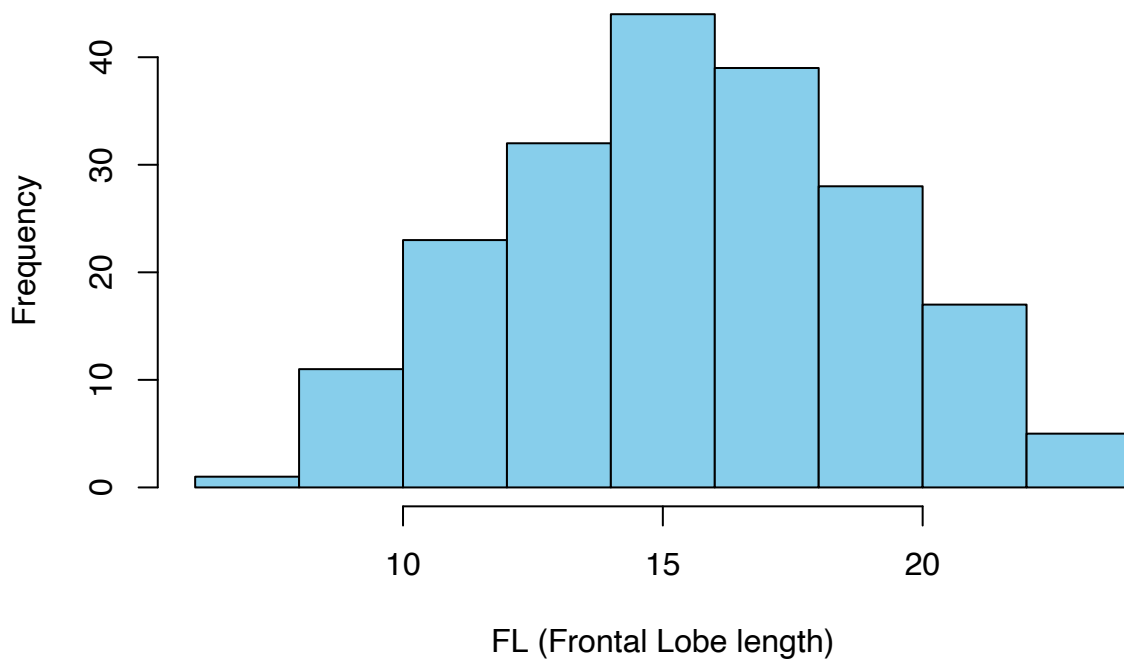
Comments :

The task involves calculating the 95% confidence interval for the grand mean of the FL feature in a specific cluster using bootstrap. The observed **grand mean** is **15.583**, which represents, in this case, the mean calculated for a specific variable (FL) across the entire dataset. The **non-pivotal confidence interval**, obtained by bootstrap, ranges from approximately **16.33478** to **16.90004**. This interval implies that the true mean for the feature in the cluster is **NOT** likely to lie within this range with 95% confidence. The **pivotal confidence interval**, ranges from about **14.26596** to **14.83122**.

The results of the data analysis reveal that both confidence intervals obtained through the bootstrap method (**non-pivotal** and **pivotal**) do not include the observed grand mean (15.583). This situation can be **interpreted** as follows. The data belonging to Cluster 1 does not adequately reflect the distribution of the entire dataset of feature FL , and therefore, it fails to represent the entire data appropriately.

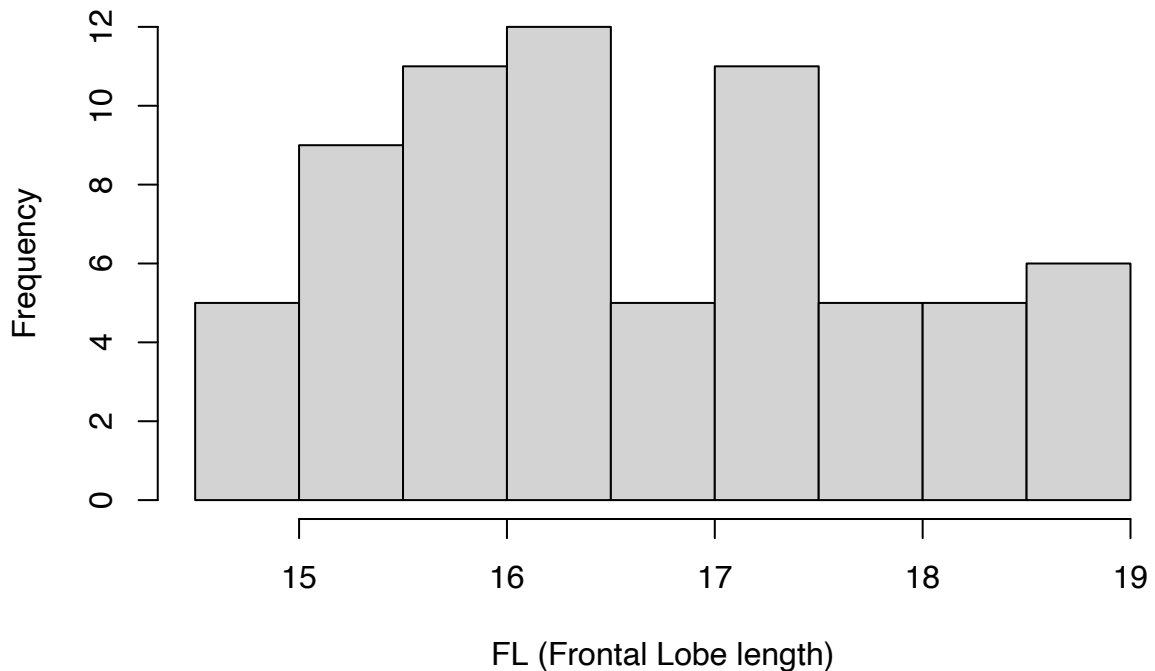
```
par(cex = 0.999)
hist(crabs$FL,
     breaks = 10,
     col = "skyblue",
     border = "black",
     main = "Frontal Lobe Length (FL) Histogram for grand mean",
     xlab = "FL (Frontal Lobe length)",
     )
```

Frontal Lobe Length (FL) Histogram for grand mean



```
par(cex = 0.999)
hist(feature_values, main = "Frontal Lobe Length (FL) Histogram for Cluster 1",
      xlab = "FL (Frontal Lobe length)"
)
```

Frontal Lobe Length (FL) Histogram for Cluster 1



Question 2

For one of the partitions created at HW 3 and a feature, compare means of the feature in two clusters using bootstrap. (Note: each application of bootstrap should be done in both versions, pivotal and non-pivotal.)

Answer 2

```
# Cluster assignment from best_kmeans_4 results generated in HW 3
clusters <- best_kmeans_4$cluster

# Extract specific feature from the entire crabs dataset (e.g., FL as the feature)
# feature_values should be extracted from the entire dataset
feature_values <- crabs[, "FL"] # Replace with the desired feature name

# Select two clusters for comparison (e.g., cluster 1 and cluster 2)
cluster1_values <- feature_values[clusters == 1]
cluster2_values <- feature_values[clusters == 2]

# Observed mean difference
obs_diff <- mean(cluster1_values) - mean(cluster2_values)
obs_diff

## [1] 6.268677

# Set bootstrap parameters
B <- 2000
boot_means <- numeric(B)

# Execute bootstrap
for (b in 1:B) {
  # Resample with replacement from each cluster
  sample_cluster1 <- sample(cluster1_values, size=length(cluster1_values), replace=TRUE)
  sample_cluster2 <- sample(cluster2_values, size=length(cluster2_values), replace=TRUE)

  # Mean difference from bootstrap samples
  boot_means[b] <- mean(sample_cluster1) - mean(sample_cluster2)
}

# Non-pivotal (Percentile) CI
ci_nonpiv <- quantile(boot_means, c(0.025, 0.975))

# Pivotal method CI
# Reflect the observed value obs_diff around the bootstrap quantiles
```

```
ci_piv <- c(2 * obs_diff - quantile(boot_means, 0.975),
           2 * obs_diff - quantile(boot_means, 0.025))

ci_nonpiv
```

```
##      2.5%      97.5%
## 5.759853 6.796269
```

```
ci_piv
```

```
##      97.5%      2.5%
## 5.741085 6.777501
```

```
# Hypothesis test: H0: Mean(cluster1) = Mean(cluster2), H1: Mean(cluster1) != Mean(cluster2)
```

```
# Non-pivotal p-value: Proportion of bootstrap differences with |obs_diff| or greater
```

```
p_value_nonpiv <- mean(abs(boot_means) >= abs(obs_diff))
```

```
# Pivotal p-value: While the pivotal CI is computed above,
```

```
# the p-value is calculated using normal approximation (as specified in the problem)
```

```
boot_mean_diff <- mean(boot_means)
```

```
boot_sd_diff <- sd(boot_means)
```

```
z_stat <- obs_diff / boot_sd_diff
```

```
p_value_piv <- 2 * pnorm(-abs(z_stat))
```

```
# Print results
```

```
print(paste("Observed difference:", obs_diff))
```

```
## [1] "Observed difference: 6.26867695184666"
```

```
print(paste("Non-pivotal p-value:", p_value_nonpiv))
```

```
## [1] "Non-pivotal p-value: 0.494"
```

```
print(paste("Pivotal p-value:", p_value_piv))
```

```
## [1] "Pivotal p-value: 5.35749135414564e-116"
```

```
# Example with significance level = 0.05
```

```
if (p_value_nonpiv < 0.05) {
```

```
  print("Non-pivotal method: Reject H0")
```

```
} else {
```

```
  print("Non-pivotal method: Fail to reject H0")
```

```
}
```

```
## [1] "Non-pivotal method: Fail to reject H0"
```

```
if (p_value_piv < 0.05) {  
  print("Pivotal method: Reject H0")  
} else {  
  print("Pivotal method: Fail to reject H0")  
}
```

```
## [1] "Pivotal method: Reject H0"
```

Comments :

The goal of this analysis is to determine whether the means of a specific feature (FL) between two clusters (Cluster 1 and Cluster 2) differ. The null hypothesis (H_0) is that “the **means of the two clusters are equal** (mean difference = 0)”, while the alternative hypothesis (H_1) is that “the **means of the two clusters are different** (mean difference $\neq 0$)”. Examining the results, the observed **mean difference** is **6.268677**, which is relatively large. This indicates that, in the actual data, the mean of Cluster 1 is about 6.27 higher than that of Cluster 2. Furthermore, the **non-pivotal** bootstrap confidence interval suggests that **<1>** the **mean difference likely falls between 5.759853 and 6.796269** , and **<2>** a **range that does not include zero**. This can be interpreted as **<1>** the calculated confidence interval is likely to include the true difference in population means and **<2>** a mean difference of zero is unlikely. This is consistent with the results from the confidence interval calculated using the **pivotal** method.

However, the situation becomes more complex when examining the **p-values**. The **non-pivotal** method produces a p-value of approximately **0.511**, which is much larger than the conventional significance threshold of 0.05. As a result, under this criterion, the conclusion would be that “the **difference** in means is **NOT statistically significant**”. On the other hand, the p-value calculated using the **pivotal** method is an extremely small value. According to this criterion, the null hypothesis can be rejected, leading to the conclusion that “the means of the two clusters are **significantly different**”.

Why does this difference occur ? The **non-pivotal** p-value directly compares the observed value to bootstrap resamples to evaluate how frequently similar or more extreme values occur. If the bootstrap distribution is concentrated near the observed value, the resulting p-value can be relatively large. This suggests that the **observed mean difference (6.268677) is not particularly extreme within the context of the bootstrap distribution**. This may reflect the fact that the two clusters genuinely represent groups with considerably different characteristics. However, this approach does not involve a direct comparison to the null hypothesis of zero mean difference. Instead, it primarily assesses “how extreme the observed value is within the bootstrap distribution itself,” which may already be shifted away from zero. **In contrast** , the **pivotal** p-value approximates the bootstrap distribution as a normal distribution and evaluates how far the observed value deviates from the null hypothesis of zero mean difference using a z-score. In this case, the large observed difference places the value in the extreme tails of the null hypothesis-centered distribution, resulting in a very small p-value.

In summary, the apparent contradiction arises from the differences in how the two methods calculate and interpret p-values. Both confidence intervals demonstrate that the mean difference is far from zero. The **pivotal p-value** strongly supports the conclusion that “the **two means are statistically significantly different**”. Ultimately, deciding which approach to rely on depends on which assumptions and methods are considered more appropriate. The higher **non-pivotal p-value** can be **attributed to a bootstrap distribution already shifted toward larger mean differences, causing the observed value to appear less extreme relative to this distribution**.

Question 3

For one of the partitions created at HW 3 and a feature, take that cluster in which the within-cluster mean is the nearest to the grand mean and test the hypothesis that these two means coincide by using bootstrap. (Note: each application of bootstrap should be done in both versions, pivotal and non-pivotal.)

Answer 3

```
### Identify the cluster whose mean of FL is closest to the grand mean
```

```
clusters <- best_kmeans_4$cluster
cluster_means <- tapply(feature_values, clusters, mean)
abs_diff <- abs(cluster_means - grand_mean)
closest_cluster <- which.min(abs_diff)

closest_cluster
```

```
## 1
## 1
```

```
cluster_mean_closest <- cluster_means[closest_cluster]
cluster_mean_closest
```

```
##      1
## 16.62029
```

```
### Hypothesis test
```

```
# H0: Cluster mean = Grand mean
```

```
#
```

```
observed_diff <- cluster_mean_closest - grand_mean
observed_diff
```

```
##      1
## 1.03729
```

```
#
```

```
cluster_values <- feature_values[clusters == closest_cluster]
```

```
### Bootstrap sampling for the difference in means (Cluster mean - Grand mean)
```

```
B <- 2000 # B
```

```
boot_means <- numeric(B)
```

```
for (b in 1:B) {
```

```

sample_values <- sample(cluster_values, size=length(cluster_values), replace=TRUE)
boot_mean_val <- mean(sample_values)

#
boot_means[b] <- boot_mean_val - grand_mean
}

### Non-pivotal (Percentile) method CI
ci_nonpiv <- quantile(boot_means, c(0.025, 0.975))

### Pivotal method CI
# observed_diff
ci_piv <- c(2 * observed_diff - quantile(boot_means, 0.975),
           2 * observed_diff - quantile(boot_means, 0.025))

ci_nonpiv

```

```

##      2.5%      97.5%
## 0.7676884 1.3126522

```

```
ci_piv
```

```

##      1      1
## 0.7619275 1.3068913

```

```

### p-value
# Non-pivotal p-value:
p_value_nonpiv <- mean(abs(boot_means) >= abs(observed_diff))

# Pivotal p-value:      (      )
p_value_piv <- mean(abs(boot_means) >= abs(observed_diff))

###
print(paste("Observed difference:", observed_diff))

```

```
## [1] "Observed difference: 1.03728985507246"
```

```
print(paste("Non-pivotal p-value:", p_value_nonpiv))
```

```
## [1] "Non-pivotal p-value: 0.509"
```

```
print(paste("Pivotal p-value:", p_value_piv))
```

```
## [1] "Pivotal p-value: 0.509"
```

```
###      (   alpha = 0.05  )
if (p_value_nonpiv < 0.05) {
  print("Non-pivotal method: Reject H0")
} else {
  print("Non-pivotal method: Fail to reject H0")
}
```

```
## [1] "Non-pivotal method: Fail to reject H0"
```

```
if (p_value_piv < 0.05) {
  print("Pivotal method: Reject H0")
} else {
  print("Pivotal method: Fail to reject H0")
}
```

```
## [1] "Pivotal method: Fail to reject H0"
```

Comments :

The hypothesis test was designed to examine whether the mean of a chosen cluster is different from the grand mean (H_0 : within-cluster mean = grand mean, H_1 : within-cluster mean $\neq$ grand mean), using bootstrap applied in both pivotal and non-pivotal versions. The observed difference is **1.03729**, and both methods produce confidence intervals that comfortably include this value. In each case, the p-value is 0.509, indicating that such a difference is easily attributable to chance. Consequently, **there is no evidence to reject the hypothesis that the cluster mean equals the grand mean** under either the **pivotal** or **non-pivotal** approach.

Epilogue

1. Appreciation

First and foremost, I want to express my deep gratitude to you, professor **Mirkin Boris Grigorevich**, for giving me the opportunity to conduct this research.

Specifically, I would like to thank you for thoroughly evaluating whether the data was suitable for this project during the preparation process, before I officially began working on it. I fully understand that improper preparation of the raw data could have caused significant difficulties during the analysis process. Thanks to your guidance and instructions, I was able to proceed with the data analysis relatively smoothly.

Secondly, I am very grateful for your excellent lectures, where you shared practical examples and gave both theoretical and practical advice based on your deep experience. I am pretty sure that this has been a great help not only to me but also to my classmates.

Lastly, thank you for delivering excellent lectures clearly and concisely within the scheduled class time. Even before taking this class, I always believed that a great professor is someone who can start class five minutes late and finish five minutes early. I believe you successfully conveyed the key points of the course to the students without extending the class time unnecessarily.

2. Key Considerations in Writing this Report

2 - 1. Interpretability matters

While writing this report, I made every effort to ensure it is as intuitive and easy to understand as possible for the reader. For instance, I used different font **colors**, **bold**, *italics*, underlines, and even capitalized certain words (for example **NOT**) to emphasize important points. These visual cues were added to help the reader quickly and intuitively grasp the main ideas of the report. Just as you value your students' time, I also understand how precious your time is.

2 - 2. Cooperation also matters

If I had to choose between a `<1>` programmer who writes 98% accurate code but in a way that no one else can understand, and a `<2>` programmer who writes 50% accurate code but in a way that anyone can easily understand, I would prefer the latter. Although **I have made my best effort to write this report based on accurate code and a proper understanding**. I believe one of the key concepts to understanding today's world is "cooperation across borders". This applies not only to academia but also to the broader industry and various other fields. I am convinced that while programmers themselves may not need sociability, the code they write must certainly have sociability. I believe such coding habits come from the practice of considering how to make oneself easily understood by others in everyday communication, whether in writing or speaking.

2 - 3. Criticism is always welcome

The reason for writing intuitive and easy-to-understand code for other programmers is not just to facilitate smoother collaboration with them. I believe it also provides a **valuable opportunity to receive critical advice** from skilled programmers, which can help improve my own shortcomings. I understand that great professors are always very busy, but if you could spare some time to give me valuable feedback, it would truly be an honor. I am confident that it would greatly contribute to improving my still-developing skills.

3. Why R instead of Python ?

I would firstly like to thank you for giving me the opportunity to work on this project using various programming languages. I was especially delighted to see that **R** , the language I used for this report, was included among the examples of programming languages you suggested for this project.



First of all, **R** is the first programming language I learned. So, I prefer **R**. In addition, through this opportunity, I learned about **R Markdown** using **L^AT_EX** , which has enabled me to use **RStudio** more effectively. **R Markdown** makes it easy to manage **updates in real time** when writing reports on data analysis results, as it allows you to easily **incorporate code and its output into the report**. This also enhances the strength of **RStudio** in **data visualization**, making it even more effective. **R** is widely used not only in my undergraduate major, **Economics** , and my previous master's major, **Bio-statistics** , but also in fields like **Econometrics**, **Genomics** , **Demography** , **Statistics** , **Sociology** , among others.

Lastly, Not just because **R** is the first programming language for me but because I am convinced that **R** also has clear advantages over **Python** when it comes to efficiently handling **data analysis** and **visualization** for datasets of a certain size in a certain field of studies in a concise and sophisticated way.

Below, I have provided simple regression analysis code in **R** and **Python** , in that order.

R codes for simple Linear Regression

```
head(mtcars)

model <- lm(mpg ~ wt, data = mtcars)

summary(model)

plot(mtcars$wt, mtcars$mpg, main = "Linear Regression in R",
      xlab = "Weight", ylab = "Miles per Gallon")
abline(model, col = "blue")
```

Python codes for simple Linear Regression

```
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

mtcars = sm.datasets.get_rdataset("mtcars", "datasets").data

X = mtcars["wt"]
y = mtcars["mpg"]

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())

plt.scatter(mtcars["wt"], mtcars["mpg"], label="Data points")
plt.plot(mtcars["wt"], model.predict(X), color="blue", label="Regression line")
plt.title("Linear Regression in Python")
plt.xlabel("Weight")
plt.ylabel("Miles per Gallon")
plt.legend()
plt.show()
```